

Deadline: Friday, Sept 11, 2026 at 11:59pm

Early turn in bonus deadline: Wednesday, Sept 9, 2026 at 11:59pm

Submission Instructions

Perform a git pull on your CIE repository. This will retrieve the hw1 directory, which will be located inside your homework directory. The hw1 directory contains starter code to help you get started. As you work on Homework 1, you should regularly commit your changes to your local Git repository and push your commits to the remote GitLab repository. The TAs will pull your remote repository to grade your homework, so make sure that your final version of hw1 has been committed and pushed before the submission deadline.

Homework Goal: Learn how to write a C++ program using file I/O, strings, vectors, and functions (optional).

In this homework, you will write a program to read data values from a file and plot them in a svg formatted file. A SVG Tutorial can be found here (<https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial>)

Example: example_graph.svg file.

```
<svg version="1.1" width="300" height="300" xmlns="http://www.w3.org/2000/svg">
  <polyline
    fill="none"
    stroke="#0074d9"
    stroke-width="3"
    points="
      0,120
      20,60
      40,80
      60,20
      80,80
      100,60
      120,150"/>
  <text x="0" y="20" font-family="Verdana" font-size="16" fill="blue">Hello!</text>
</svg>
```

Image produced by example_graph.svg when viewed in CLion.



The following are the minimum specifications for your program.

1. Write a C++ program that performs the following tasks:
 - a. [15pts] Read data values from the file and store them in a C++ vector. Then close the file before computing statistics.

The file format is the following: The file starts with a '[' character followed by text. The first data value starts after the second '['. Each data value is followed immediately by a comma and a space. The last data value is immediately followed by a ']' character. You may ignore all the characters between the first two '[' characters and after the first ']' character. The data is output from some of my previous Peloton rides. The values are sampled every second and are in units of Watts.

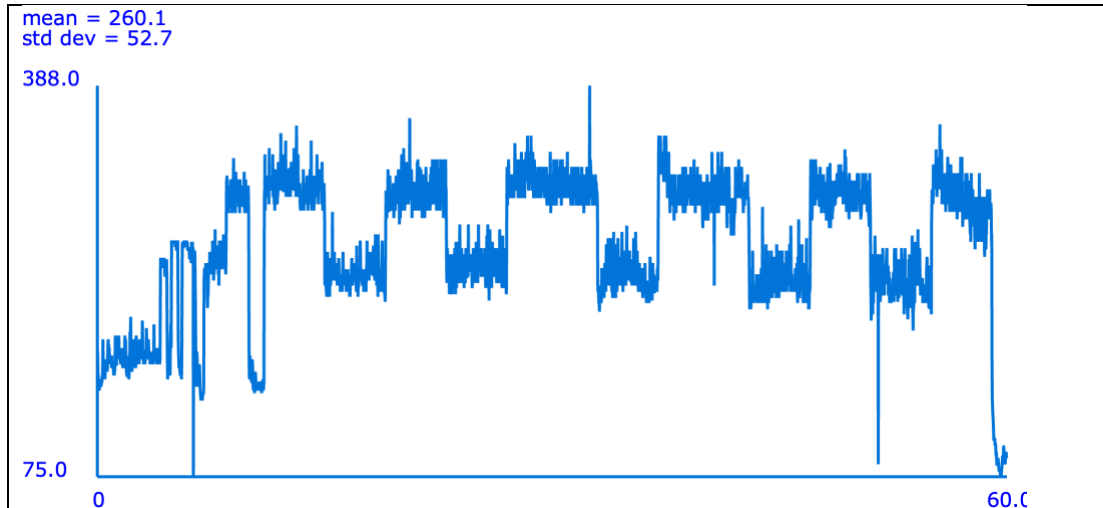
The following lines of code near the top of main.

```
ifstream fin("ride_data.txt");  
//ifstream fin("ride_data2.txt");  
//ifstream fin("ride_data3.txt");  
//ifstream fin("ride_data4.txt");  
//ifstream fin("ride_data5.txt");
```

Only one of the five lines of code should be uncommented. Change which line is uncommented and recompile to process each input file.

- b. [15pts] Compute and store the following statistics in variables: mean, standard deviation, minimum, and maximum of the data.
 - c. [20pts] Open a file called "output_graph.svg". Write svg commands to the file to draw the x- and y- axes and label the min and max values on each axis. Then write the mean and standard deviation on the graph. Then close the file. Your program and output file must satisfy the following constraints.
 - a) Your program should define and use variables to store the width and height of the image. You should hard code these values (e.g., 1000 wide and 500 high) and then use these variables throughout the rest of your program. Your program should produce a reasonable plot if these values are changed, i.e., 512 wide by 200 high or 400 wide by 600 high. The TAs will change these values when they grade your homework.
 - b) Your program should define variables that store the top, bottom, left and right margins (e.g., top=20, bottom=60, left=80, right=50). You should select values to give you room to display the y-min, y-max, x-min, and x-max values. The TAs will change these values when they grade your homework.
 - c) The x-axis should be drawn along the bottom margin and not extend into the left and right margins. The y-axis should be drawn along the left margin and not extend into the top and bottom margins.
 - d) You may hard code the small offsets to display the y-min and y-max to the left of the y-axis and x-min and x-max below the x-axis. (See example below.)
 - e) The mean and standard deviation should be displayed at the top left corner of the image within the top margin. (See example below.)
 - d. [20pts] Modify your code from part c to plot the graph of the data.
 2. [30pts] Fully document your program.

Example output for the file *ride_data.txt*:



Notes:

1. List of header files that you need for your program.

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <cmath>
```

2. The following lines appear in the *CMakeLists.txt* file provided to you. These commands will copy the files from your source directory to the build directory (i.e., *cmake-build-debug*). Your program reads files from the *cmake-build-debug* directory and not your source directory. If you change the contents of a file in your source directory, then you need to reload your *CMakeLists.txt* file to copy the new file to the *cmake-build-debug* directory.

```
configure_file(ride_data.txt ride_data.txt COPYONLY)
configure_file(ride_data2.txt ride_data2.txt COPYONLY)
configure_file(ride_data3.txt ride_data3.txt COPYONLY)
configure_file(ride_data4.txt ride_data4.txt COPYONLY)
configure_file(ride_data5.txt ride_data5.txt COPYONLY)
```

3. It is optional to write functions to compute the *average*, *standard deviation*, etc. However, if you write functions then each function must be documented as specified by the CIE style guide.
4. The *getline()* command below may be useful. In this example, *getline()* reads characters from the input file stream *fin* into the *tmp* string until the terminating character '[' is read. The '[' is read but is not stored in *tmp*. You can change the terminating character '[' to any character such as a comma. The command will read all the characters until it finds the terminating character.

```
string tmp;
getline(fin, tmp, '[');
```

5. Example code for writing a string literal that includes special characters like double quotes to the file stream called *fout*.

```
fout << R"(<svg version="1.1" width="512" height="512" );";  
fout << R"(xmlns="http://www.w3.org/2000/svg">) " << endl;
```

6. The output of your program will be written to the *cmake-build-debug* directory. Do not edit files in the *cmake-build-debug* directory.
7. **The TAs will evaluate your program using different files than the ones that you used to develop your program. Thus, your program should work for general input data and not only for the files that were provided for you.**